МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № 3

по дисциплине «Математические Основы Баз Данных»

по теме: «Многотабличные запросы и аналитические представления»

Выполнил студент группы:
БВТ2505
Коновалова Варвара Вячеславовна

Проверил:
Быватов Семён Юрьевич

Москва 2026

Оглавление

Введение 4

Выполнение 5

Заключение 24

Введение

Цель и задачи работы:

Целью данной лабораторной работы является приобретение практических навыков составления многотабличных и аналитических SQL-запросов в СУБД PostgreSQL: соединения таблиц, агрегирования и группировки, подзапросов, общих табличных выражений (в том числе рекурсивных) и создания представлений, а также закрепление знаний о типах соединений, порядке выполнения секций WHERE, GROUP BY и HAVING и различиях между обычными и материализованными представлениями.

Задачи:

1. Составить запрос с внутренним соединением не менее трёх таблиц (товар - категория - бренд).
2. Составить запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице (например, товары без отзывов).
3. Построить запрос с группировкой и агрегацией: по каждой категории - количество товаров и средняя цена.
4. Применить HAVING для фильтрации групп (бренды с выручкой выше заданной).
5. Написать по одному запросу с подзапросом каждого вида: скалярным, с IN и с EXISTS (коррелированным).
6. Использовать CTE для разбиения сложного запроса на шаги и реализовать рекурсивный CTE для обхода дерева категорий.
7. Создать не менее двух представлений для типовых отчётов и продемонстрировать обращение к ним.
8. Для каждого запроса привести текст, назначение и фрагмент результата.

Выполнение

Исходные данные и допущения

Работа выполняется на базе данных «Онлайн-зоомагазин», спроектированной в лабораторной работе № 1 и наполненной тестовыми данными в лабораторной работе № 2: 13 таблиц, в том числе 1000 товаров, 1200 заказов, 2400 позиций заказов и 600 отзывов. Для воспроизведения результата скрипты выполняются по порядку: 01_schema.sql (схема), 02_seed.sql (наполнение, setseed(0.42) фиксирует случайные значения) и 03_lab3.sql (запросы и представления данной работы). Скрипт 03_lab3.sql можно запускать повторно: перед созданием представления удаляются через DROP ... IF EXISTS.

Особенность схемы: отдельной таблицы продавцов в ней нет - каждый товар принадлежит бренду (products.brand_id -> brands.id). Поэтому цепочка «товар - категория - продавец» из задания реализуется как «товар - категория - бренд», а аналитика «по продавцам» строится по брендам. Выручка вычисляется по позициям заказов как SUM(quantity * unit_price); отменённые заказы (status = 'cancelled') в расчёт не входят.

Связи таблиц, используемые в запросах работы, приведены ниже.

Связь (внешний ключ)	Тип связи	Где используется
products.category_id -> categories.id	N:1	запросы 1, 3, 4а, 7в, 7б
products.brand_id -> brands.id	N:1	запросы 1, 4, 8, представления
categories.parent_id -> categories.id	N:1 (самоссылка)	запросы 1а, 9, 9а
reviews.product_id -> products.id	N:1	запросы 2, 2а, представление 2
order_items.order_id -> orders.id	N:1	запросы 4, 6, 8, представление 1
order_items.product_id -> products.id	N:1	запросы 4, 6, 8, представление 1
orders.user_id -> users.id	N:1	запрос 2б
pets.user_id -> users.id	N:1	запрос 7

Связь (внешний ключ)	Тип связи	Где используется
stock.product_id -> products.id	1:1	представление 2

1. Напишите запрос с внутренним соединением не менее трёх таблиц (товар - категория - продавец).

1. Внутреннее соединение (INNER JOIN)

INNER JOIN оставляет только те пары строк, для которых выполняется условие в секции ON - обычно равенство внешнего ключа одной таблицы и первичного ключа другой. Строки без соответствия отбрасываются. Таблицам удобно давать короткие псевдонимы (AS p, AS c, AS b) и обращаться к столбцам через них.

Запрос 1. Товар - категория - бренд

Текст запроса:

```
-- Запрос 1
SELECT p.id,
       p.title       AS "Товар",
       c.name        AS "Категория",
       b.name        AS "Бренд",
       b.country     AS "Страна",
       p.price       AS "Цена"
FROM products   AS p
JOIN categories AS c ON c.id = p.category_id
JOIN brands     AS b ON b.id = p.brand_id
WHERE p.is_active
ORDER BY p.id
LIMIT 10;
```

Назначение:

Соединяет три таблицы: products с categories (по category_id) и products с brands (по brand_id). INNER JOIN оставляет только товары, для которых найдены и категория, и бренд; благодаря NOT NULL и внешним ключам, заданным в первой работе, строки не теряются. Выводятся активные товары, для компактности - первые 10 строк.

Фрагмент результата:

	id bigint	Товар text	Категория text	Бренд text	Страна text	Цена numeric (12,2)
1	1	Лежанка «Classic» №1	Лежанки	Royal Can…	Франция	3200.31
2	2	Когтеточка «Light» №2	Когтеточки	Royal Can…	Франция	3310.15
3	3	Мяч «Sensitive» №3	Мячи и фрисби	Beaphar	Нидерланд…	2781.39
4	4	Сухой корм «Active» №4	Сухой корм	Pedigree	США	527.80
5	5	Сухой корм «Premium» …	Сухой корм	Beaphar	Нидерланд…	1366.08
6	6	Шампунь «Classic» №6	Шампуни и космети…	Hill's	США	4715.86
7	7	Шампунь «Light» №7	Шампуни и космети…	Beaphar	Нидерланд…	4916.12
8	8	Шампунь «Sensitive» №8	Шампуни и космети…	Beaphar	Нидерланд…	4550.87
9	9	Лежанка «Active» №9	Лежанки	Pro Plan	Франция	1646.51
10	11	Сухой корм «Classic» №…	Сухой корм	Hartz	США	4100.20

Строк в результате: 10. Показаны первые 10 строк (LIMIT 10).

Запрос 1а (дополнение). Самосоединение: подкатегория и её родитель

Текст запроса:

```
-- Запрос 1а (дополнение). Четыре таблицы + самосоединение:
подкатегория и её родитель
SELECT p.id,
       p.title   AS "Товар",
       c.name    AS "Подкатегория",
       pc.name   AS "Корневая категория",
       b.name    AS "Бренд"
FROM products   AS p
JOIN categories AS c  ON c.id  = p.category_id
JOIN categories AS pc ON pc.id = c.parent_id
JOIN brands     AS b  ON b.id  = p.brand_id
ORDER BY p.id
LIMIT 10;
```

Назначение:

Четвёртое соединение - самосоединение таблицы categories: c - подкатегория товара, pc - её родительская категория (pc.id = c.parent_id). Так для каждого товара видна вся ветка иерархии из первой работы. Товары привязаны только к подкатегориям, у которых родитель всегда есть, поэтому INNER JOIN строк не теряет.

Фрагмент результата:

	id bigint	Товар text	Подкатегория text	Корневая категория text	Бренд text
1	1	Лежанка «Classic» №1	Лежанки	Лежанки и домики	Royal Can…
2	2	Когтеточка «Light» №2	Когтеточки	Игрушки	Royal Can…
3	3	Мяч «Sensitive» №3	Мячи и фрисби	Игрушки	Beaphar
4	4	Сухой корм «Active» №4	Сухой корм	Корма	Pedigree
5	5	Сухой корм «Premium» …	Сухой корм	Корма	Beaphar
6	6	Шампунь «Classic» №6	Шампуни и косметі…	Гигиена и уход	Hill's
7	7	Шампунь «Light» №7	Шампуни и косметі…	Гигиена и уход	Beaphar
8	8	Шампунь «Sensitive» №8	Шампуни и косметі…	Гигиена и уход	Beaphar
9	9	Лежанка «Active» №9	Лежанки	Лежанки и домики	Pro Plan
10	10	Лежанка «Premium» №10	Лежанки	Лежанки и домики	Whiskas

Строк в результате: 10. Показаны первые 10 строк.

2. Напишите запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице (например, товары без отзывов).

2. Левое внешнее соединение (LEFT JOIN)

LEFT JOIN сохраняет все строки левой таблицы; если соответствия справа нет, вместо столбцов правой таблицы подставляется NULL. Условие IS NULL по столбцу правой таблицы в WHERE оставляет только строки без соответствий. Это ключевое отличие от INNER JOIN, который такие строки теряет.

Запрос 2. Товары без отзывов

Текст запроса:

```
-- Запрос 2. Товары без отзывов
SELECT p.id,
       p.title AS "Товар",
       p.price AS "Цена"
FROM products AS p
LEFT JOIN reviews AS r ON r.product_id = p.id
WHERE r.id IS NULL
ORDER BY p.id
LIMIT 10;
```

Назначение:

LEFT JOIN сохраняет все товары; если у товара нет отзывов, столбцы reviews заполняются NULL. Условие r.id IS NULL оставляет только такие товары (приём «anti-join»). В базе 600 отзывов относятся к 600 разным товарам, поэтому без отзывов остаются 400 товаров из 1000.

Фрагмент результата:

	id [PK] bigint	Товар text	Цена numeric (12,2)
1	1	Лежанка «Classic» №1	3200.31
2	6	Шампунь «Classic» №6	4715.86
3	7	Шампунь «Light» №7	4916.12
4	13	Мяч «Sensitive» №13	4730.09
5	14	Мяч «Active» №14	475.94
6	20	Ошейник «Premium» №20	4211.88
7	21	Лежанка «Classic» №21	4419.64
8	27	Лежанка «Light» №27	631.30
9	28	Сухой корм «Sensitive» №...	333.66
10	34	Сухой корм «Active» №34	3107.96

Строк в результате: 10. Показаны первые 10 строк из 400.

Запрос 2а (дополнение). Подсчёт товаров с отзывами и без

Текст запроса:

```
-- Запрос 2а (дополнение). Сколько товаров с отзывами и без
SELECT COUNT(*)                    AS "Всего товаров",
       COUNT(r.id)                 AS "Строк с отзывом",
       COUNT(*) FILTER (WHERE r.id IS NULL) AS "Без отзывов"
FROM products AS p
LEFT JOIN reviews AS r ON r.product_id = p.id;
```

Назначение:

Демонстрирует разницу между COUNT(*) и COUNT(столбец): COUNT(*) считает все строки результата соединения (1000), COUNT(r.id) не учитывает NULL и считает только строки, где отзыв нашёлся (600). Конструкция FILTER (WHERE r.id IS NULL) считает строки без отзыва (400).

Фрагмент результата:

	Всего товаров bigint	Строк с отзывом bigint	Без отзывов bigint
1	1000	600	400

Строк в результате: 1. Результат - одна строка: 1000 = 600 + 400.

Запрос 2б (дополнение). Пользователи без заказов

Текст запроса:

```
-- Запрос 2б (дополнение). Пользователи без заказов
SELECT u.id, u.last_name || ' ' || u.first_name AS "Пользователь", u.email
FROM users AS u
LEFT JOIN orders AS o ON o.user_id = u.id
WHERE o.id IS NULL
ORDER BY u.id;
```

Назначение:

Тот же приём применён к другой паре таблиц: LEFT JOIN users с orders и фильтр o.id IS NULL. Находит зарегистрированных пользователей, которые ни разу не оформляли заказ. INNER JOIN таких пользователей потерял бы, поэтому здесь нужен именно LEFT JOIN.

Фрагмент результата:

	id [PK] bigint	Пользователь text	email text
1	76	Петров Дмитрий	user76@example.com
2	88	Попов Сергей	user88@example.com
3	138	Иванов Сергей	user138@example.co…
4	169	Иванова Натал…	user169@example.co…

Строк в результате: 4. В выборке 4 пользователя из 300.

3. Постройте запрос с группировкой и агрегацией: по каждой категории - количество товаров и средняя цена.

3. Группировка и агрегатные функции

Секция GROUP BY делит строки на группы по значениям перечисленных столбцов, а агрегатные функции (COUNT, SUM, AVG, MIN, MAX) возвращают по одному значению на группу. В списке SELECT допустимы только столбцы из GROUP BY и агрегатные выражения.

Запрос 3. Количество товаров и средняя цена по категориям

Текст запроса:

```
-- Запрос 3
SELECT c.name                    AS "Категория",
       COUNT(*)                  AS "Товаров",
       round(AVG(p.price), 2)    AS "Средняя цена",
       MIN(p.price)              AS "Мин. цена",
       MAX(p.price)              AS "Макс. цена"
FROM products   AS p
JOIN categories AS c ON c.id = p.category_id
GROUP BY c.name
ORDER BY "Товаров" DESC;
```

Назначение:

Группировка по названию категории: для каждой группы вычисляются количество товаров, средняя, минимальная и максимальная цена. Функция round(..., 2) округляет среднюю цену до копеек. Сортировка выполняется по псевдониму «Товаров» - ORDER BY выполняется после SELECT, поэтому псевдоним ему доступен. Сумма товаров по всем категориям равна 1000.

Фрагмент результата:

	category_id bigint	category text	products_count bigint	avg_price numeric
1	6	Сухой корм	161	2544.93
2	12	Лежанки	160	2565.43
3	11	Шампуни и косметик...	149	2535.21
4	7	Влажный корм	147	2715.03
5	8	Мячи и фрисби	147	2440.91
6	10	Ошейники и поводки	126	2597.58
7	9	Когтеточки	110	2484.80

Строк в результате: 7. Семь строк - по одной на каждую подкатегорию.

4. Примените HAVING для фильтрации групп (например, продавцы с выручкой выше заданной).

4. Фильтрация групп: HAVING

WHERE отбирает исходные строки до группировки, а HAVING фильтрует уже сформированные группы по результату агрегации. Поэтому условие по SUM или COUNT можно записать только в HAVING.

Запрос 4. Бренды с выручкой выше 1 000 000

Текст запроса:

```
-- Запрос 4. Бренды с выручкой выше 1 000 000 (без отменённых заказов)
SELECT b.name                          AS "Бренд",
       COUNT(DISTINCT o.id)            AS "Заказов",
       SUM(oi.quantity * oi.unit_price)  AS "Выручка"
FROM brands      AS b
JOIN products    AS p  ON p.brand_id = b.id
JOIN order_items AS oi ON oi.product_id = p.id
JOIN orders      AS o  ON o.id = oi.order_id
WHERE o.status <> 'cancelled'
GROUP BY b.name
HAVING SUM(oi.quantity * oi.unit_price) > 1000000
ORDER BY "Выручка" DESC;
```

Назначение:

Выручка бренда считается по позициям заказов (количество × цена на момент покупки). Условие o.status <> 'cancelled' в WHERE отбирает строки до группировки, а условие по сумме выручки записано в HAVING - после группировки. COUNT(DISTINCT o.id) считает именно заказы, а не позиции. Из десяти брендов порог преодолевают четыре.

Фрагмент результата:

	Бренд text	Заказов bigint	Выручка numeric
1	Четвероногий др...	212	1108642.49
2	Hartz	184	1071220.57
3	Pedigree	170	1051793.12
4	Trixie	191	1030752.43

Строк в результате: 4. Четыре строки; бренды упорядочены по убыванию выручки.

Запрос 4а (дополнение). HAVING по количеству

Текст запроса:

```
-- Запрос 4а (дополнение). HAVING по количеству: категории, где больше
140 товаров
SELECT c.name AS "Категория", COUNT(*) AS "Товаров"
FROM products AS p
```

```
JOIN categories AS c ON c.id = p.category_id
GROUP BY c.name
HAVING COUNT(*) > 140
ORDER BY "Товаров" DESC;
```

Назначение:

Группы можно фильтровать и по COUNT: выводятся категории, в которых больше 140 товаров. Категории «Ошейники и поводки» (126) и «Когтеточки» (110) в выборку не попадают.

Фрагмент результата:

	Категория text	Товаров bigint
1	Сухой корм	161
2	Лежанки	160
3	Шампуни и косметик...	149
4	Влажный корм	147
5	Мячи и фрисби	147

Строк в результате: 5. Пять строк из семи.

5. Напишите по одному запросу с подзапросом каждого вида: скалярным, с IN и с EXISTS (или коррелированным).

5. Подзапросы

Подзапрос - запрос, вложенный в другой запрос. Скалярный возвращает ровно одно значение, подзапрос в IN - множество значений, EXISTS проверяет, вернул ли подзапрос хотя бы одну строку. Некоррелированный подзапрос вычисляется один раз, коррелированный ссылается на столбцы внешнего запроса и логически выполняется для каждой его строки.

Запрос 5. Скалярный подзапрос

Текст запроса:

```
-- Запрос 5. Скалярный подзапрос: товары дороже средней цены по базе
SELECT id, title, price,
      round((SELECT AVG(price) FROM products), 2) AS avg_price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
```

```
ORDER BY price DESC
LIMIT 10;
```

Назначение:

Подзапрос (SELECT AVG(price) FROM products) возвращает единственное значение - среднюю цену по базе. Он некоррелированный и вычисляется один раз; внешний запрос оставляет товары дороже этого значения. Тот же подзапрос в списке SELECT выводит среднюю цену отдельным столбцом для наглядности.

Фрагмент результата:

	id [PK] bigint	title text	price numeric (12,2)	avg_price numeric
1	730	Влажный корм «Premium» №7…	4999.32	2556.49
2	958	Влажный корм «Sensitive» №9…	4990.02	2556.49
3	206	Когтеточка «Classic» №206	4989.66	2556.49
4	734	Лежанка «Active» №734	4976.34	2556.49
5	47	Влажный корм «Light» №47	4970.58	2556.49
6	146	Шампунь «Classic» №146	4968.75	2556.49
7	196	Ошейник «Classic» №196	4965.32	2556.49
8	745	Лежанка «Premium» №745	4960.22	2556.49
9	833	Шампунь «Sensitive» №833	4952.11	2556.49
10	343	Влажный корм «Sensitive» №3…	4933.33	2556.49

Строк в результате: 10. Показаны 10 самых дорогих товаров; средняя цена по базе - 2556.49.

Запрос 6. Подзапрос с IN

Текст запроса:

```
-- Запрос 6. IN: товары, которые покупали в доставленных заказах
SELECT id, title, price
FROM products
WHERE id IN (SELECT oi.product_id
             FROM order_items AS oi
             JOIN orders AS o ON o.id = oi.order_id
             WHERE o.status = 'delivered')
ORDER BY id
LIMIT 10;
```

Назначение:

Подзапрос возвращает множество идентификаторов товаров, которые входили в доставленные заказы (соединение order_items с orders). Внешний запрос выбирает товары, чей id входит в это множество. Подзапрос не зависит от внешнего и вычисляется один раз.

Фрагмент результата:

	id [PK] bigint	title text	price numeric (12,2)
1	2	Когтеточка «Light» №2	3310.15
2	4	Сухой корм «Active» №4	527.80
3	5	Сухой корм «Premium» ...	1366.08
4	7	Шампунь «Light» №7	4916.12
5	11	Сухой корм «Classic» №...	4100.20
6	13	Мяч «Sensitive» №13	4730.09
7	15	Ошейник «Premium» №15	3704.89
8	18	Лежанка «Sensitive» №18	357.35
9	20	Ошейник «Premium» №20	4211.88
10	21	Лежанка «Classic» №21	4419.64

Строк в результате: 10. Показаны первые 10 строк.

Запрос 7. Подзапрос с EXISTS (коррелированный)

Текст запроса:

```
-- Запрос 7. EXISTS (коррелированный): пользователи, у которых есть собака
SELECT u.id, u.last_name || ' ' || u.first_name AS "Пользователь"
FROM users AS u
WHERE EXISTS (SELECT 1 FROM pets AS pt
              WHERE pt.user_id = u.id AND pt.species = 'dog')
ORDER BY u.id
LIMIT 10;
```

Назначение:

Подзапрос ссылается на u.id внешнего запроса, то есть коррелирован: для каждого пользователя проверяется, есть ли у него хотя бы один питомец с species = 'dog'. EXISTS истинен, если подзапрос вернул хотя бы одну строку, и

прекращает поиск на первой найденной, поэтому список столбцов подзапроса (SELECT 1) значения не имеет.

Фрагмент результата:

	id [PK] bigint	Пользователь text
1	2	Петров Пётр
2	3	Иванова Мария
3	10	Кузнецов Иван
4	12	Попов Пётр
5	14	Попов Алексей
6	16	Петров Дмитрий
7	19	Иванова Наталья
8	20	Иванов Иван
9	23	Смирнова Мария
10	24	Кузнецов Алекс…

Строк в результате: 10. Показаны первые 10 строк.

Запрос 7а (дополнение). NOT EXISTS

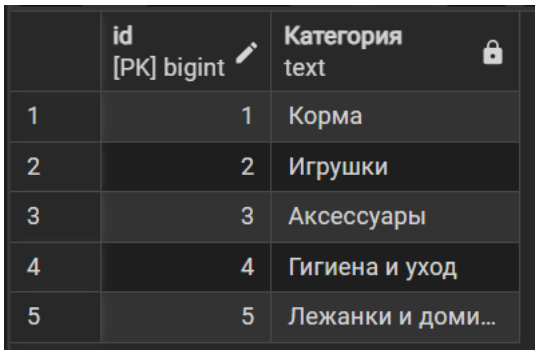
Текст запроса:

```
-- Запрос 7а (дополнение). NOT EXISTS: категории, в которых нет ни одного товара
SELECT c.id, c.name AS "Категория"
FROM categories AS c
WHERE NOT EXISTS (SELECT 1 FROM products AS p WHERE p.category_id = c.id)
ORDER BY c.id;
```

Назначение:

NOT EXISTS проверяет отсутствие связанных данных. Запрос находит категории, в которых нет ни одного товара. Это пять корневых категорий: товары привязаны только к подкатегориям (id 6–12).

Фрагмент результата:



	id [PK] bigint	Категория text
1	1	Корма
2	2	Игрушки
3	3	Аксессуары
4	4	Гигиена и уход
5	5	Лежанки и доми…

Строк в результате: 5. Пять корневых категорий.

Запрос 7б (дополнение). Коррелированный скалярный подзапрос

Текст запроса:

```
-- Запрос 7б (дополнение). Коррелированный подзапрос: цена выше
средней в своей категории
SELECT p.id, p.title, p.category_id, p.price,
      (SELECT round(AVG(p3.price), 2) FROM products AS p3
       WHERE p3.category_id = p.category_id) AS category_avg
FROM products AS p
WHERE p.price > (SELECT AVG(p2.price)
                 FROM products AS p2
                 WHERE p2.category_id = p.category_id)
ORDER BY p.id
LIMIT 10;
```

Назначение:

Подзапрос вычисляет среднюю цену в категории текущего товара (p2.category_id = p.category_id), поэтому товар сравнивается со средней ценой своей категории, а не всей базы. В списке SELECT тот же показатель выводится столбцом category_avg.

Фрагмент результата:

	id [PK] bigint	title text	category_id bigint	price numeric (12,2)	category_avg numeric
1	1	Лежанка «Classic» №1	12	3200.31	2565.43
2	2	Когтеточка «Light» №2	9	3310.15	2484.80
3	3	Мяч «Sensitive» №3	8	2781.39	2440.91
4	6	Шампунь «Classic» №6	11	4715.86	2535.21
5	7	Шампунь «Light» №7	11	4916.12	2535.21
6	8	Шампунь «Sensitive» №8	11	4550.87	2535.21
7	10	Лежанка «Premium» №10	12	3366.62	2565.43
8	11	Сухой корм «Classic» №...	6	4100.20	2544.93
9	12	Мяч «Light» №12	8	3242.27	2440.91
10	13	Мяч «Sensitive» №13	8	4730.09	2440.91

Строк в результате: 10. Показаны первые 10 строк; во всех строках цена выше category_avg.

Запрос 7в (дополнение). Подзапрос в секции FROM

Текст запроса:

```
-- Запрос 7в (дополнение). Подзапрос в FROM (производная таблица)
SELECT t.category_id, t.avg_price
FROM (SELECT category_id, round(AVG(price), 2) AS avg_price
      FROM products
      GROUP BY category_id) AS t
WHERE t.avg_price > 2550
ORDER BY t.avg_price DESC;
```

Назначение:

Подзапрос в FROM (производная таблица) считает среднюю цену по каждой категории, а внешний запрос отбирает из этого временного результата категории со средней ценой выше 2550. Производной таблице обязательно присваивается псевдоним (AS t).

Фрагмент результата:

	category_id bigint	avg_price numeric
1	7	2715.03
2	10	2597.58
3	12	2565.43

Строк в результате: 3. Три категории со средней ценой выше 2550.

6. Используйте CTE для разбиения сложного запроса на шаги; реализуйте рекурсивный CTE для обхода дерева категорий.

6. Общие табличные выражения (CTE)

Конструкция WITH даёт имя промежуточному результату, к которому основной запрос обращается как к таблице; в одном запросе можно объявить несколько CTE. Рекурсивный CTE (WITH RECURSIVE) состоит из якорной части, выбирающей начальные строки, и рекурсивной части, которая ссылается на сам CTE и на каждом шаге добавляет следующий уровень иерархии.

Запрос 8. Несколько CTE: доля бренда в общей выручке

Текст запроса:

```
-- Запрос 8. Несколько CTE: доля каждого бренда в общей выручке
WITH brand_rev AS (
    SELECT p.brand_id, SUM(oi.quantity * oi.unit_price) AS revenue
    FROM order_items AS oi
    JOIN orders   AS o ON o.id = oi.order_id
    JOIN products AS p ON p.id = oi.product_id
    WHERE o.status <> 'cancelled'
    GROUP BY p.brand_id
),
total AS (
    SELECT SUM(revenue) AS total_revenue FROM brand_rev
)
SELECT b.name AS "Бренд",
       r.revenue AS "Выручка",
       round(100 * r.revenue / t.total_revenue, 2) AS "Доля, %"
FROM brand_rev AS r
JOIN brands AS b ON b.id = r.brand_id
CROSS JOIN total AS t
ORDER BY r.revenue DESC;
```

Назначение:

Запрос разбит на логические шаги. CTE brand_rev считает выручку каждого бренда, CTE total - общую выручку. Основной запрос соединяет их с таблицей brands и вычисляет долю бренда в процентах. Без CTE пришлось бы дублировать подзапрос или применять оконную функцию.

Фрагмент результата:

	Бренд text	Выручка numeric	Доля, % numeric
1	Четвероногий др...	1108642.49	11.51
2	Hartz	1071220.57	11.12
3	Pedigree	1051793.12	10.92
4	Trixie	1030752.43	10.70
5	Ferplast	999059.98	10.37
6	Whiskas	972574.73	10.09
7	Royal Canin	914852.06	9.50
8	Pro Plan	893300.91	9.27
9	Beaphar	863802.65	8.97
10	Hill's	728461.35	7.56

Строк в результате: 10. Десять строк; сумма долей равна 100 % (с точностью до округления).

Запрос 9. Рекурсивный CTE: обход дерева категорий

Текст запроса:

```
-- Запрос 9. Рекурсивный CTE: обход дерева категорий
WITH RECURSIVE category_tree AS (
    SELECT id, parent_id, name, 1 AS depth, name::text AS path
    FROM categories
    WHERE parent_id IS NULL
    UNION ALL
    SELECT c.id, c.parent_id, c.name, ct.depth + 1, ct.path || ' > ' || c.name
    FROM categories AS c
    JOIN category_tree AS ct ON c.parent_id = ct.id
)
SELECT id, depth AS "Уровень", path AS "Путь"
FROM category_tree
ORDER BY path;
```

Назначение:

Якорная часть выбирает корневые категории (parent_id IS NULL) с уровнем 1. Рекурсивная часть соединяет categories с уже найденными строками (c.parent_id = ct.id) и увеличивает уровень на 1, достраивая путь «родитель > потомок». Рекурсия заканчивается, когда новых строк не появляется. UNION ALL объединяет шаги без удаления дубликатов. Запрос работает для дерева любой глубины, в данных - два уровня.

Фрагмент результата:

	id bigint	Уровень integer	Путь text
1	3	1	Аксессуары
2	10	2	Аксессуары > Ошейники и поводки
3	4	1	Гигиена и уход
4	11	2	Гигиена и уход > Шампуни и космети...
5	2	1	Игрушки
6	9	2	Игрушки > Когтеточки
7	8	2	Игрушки > Мячи и фрисби
8	1	1	Корма
9	7	2	Корма > Влажный корм
10	6	2	Корма > Сухой корм
11	5	1	Лежанки и домики
12	12	2	Лежанки и домики > Лежанки

Строк в результате: 12. 12 строк: 5 корневых категорий (уровень 1) и 7 подкатегорий (уровень 2); сортировка по пути ставит потомков сразу после родителя.

Запрос 9а (дополнение). Поддерево заданной категории

Текст запроса:

```
-- Запрос 9а (дополнение). Все подкатегории заданной категории и число товаров в них
WITH RECURSIVE sub AS (
    SELECT id, name FROM categories WHERE slug = 'toys'
    UNION ALL
    SELECT c.id, c.name FROM categories AS c JOIN sub ON c.parent_id = sub.id
)
```

```
SELECT sub.name AS "Категория", COUNT(p.id) AS "Товаров"
FROM sub
LEFT JOIN products AS p ON p.category_id = sub.id
GROUP BY sub.name
ORDER BY sub.name;
```

Назначение:

Якорная часть выбирает одну категорию по slug ('toys'), рекурсивная - всех её потомков. LEFT JOIN с products считает товары в каждой найденной категории и сохраняет корневую категорию «Игрушки», у которой собственных товаров нет.

Фрагмент результата:

	Категория text	Товаров bigint
1	Игрушки	0
2	Когтеточки	110
3	Мячи и фрис…	147

Строк в результате: 3. Категория «Игрушки» и две её подкатегории.

7. Создайте не менее двух представлений для типовых отчётов и продемонстрируйте обращение к ним.

7. Представления (VIEW)

Обычное представление хранит в базе не данные, а текст запроса: при каждом обращении выполняется лежащий в основе SELECT, поэтому возвращаются актуальные данные. Представления скрывают сложность соединений и дают готовые витрины для отчётов. Создано два обычных представления и, дополнительно, одно материализованное, которое физически хранит результат запроса.

Представление 1. v_brand_revenue - выручка по брендам

Текст запроса:

```
-- Представление 1. Выручка по брендам (без отменённых заказов)
CREATE VIEW v_brand_revenue AS
SELECT b.id                         AS brand_id,
       b.name                       AS brand_name,
       COUNT(DISTINCT o.id)         AS orders_count,
```

```
    SUM(oi.quantity)              AS items_sold,
    SUM(oi.quantity * oi.unit_price)  AS revenue
FROM brands      AS b
JOIN products    AS p  ON p.brand_id = b.id
JOIN order_items AS oi ON oi.product_id = p.id
JOIN orders      AS o  ON o.id = oi.order_id
WHERE o.status <> 'cancelled'
GROUP BY b.id, b.name;
```

Назначение:

Сохраняет запрос с четырьмя соединениями и группировкой: для каждого бренда - число заказов, число проданных единиц и выручка без отменённых заказов. Сложный запрос теперь вызывается коротким SELECT из представления.

Фрагмент результата:

Команда CREATE VIEW выполнена; обращение к представлению - в запросе 10.

Представление 2. v_product_catalog — витрина каталога
Текст запроса:

```
-- Представление 2. Витрина каталога: товар, категория, бренд, остаток, рейтинг
CREATE VIEW v_product_catalog AS
SELECT p.id,
    p.title,
    c.name              AS category,
    b.name              AS brand,
    p.price,
    COALESCE(s.quantity, 0)   AS in_stock,
    round(AVG(r.score), 2)    AS avg_score,
    COUNT(r.id)               AS reviews_count
FROM products   AS p
JOIN categories AS c ON c.id = p.category_id
JOIN brands     AS b ON b.id = p.brand_id
LEFT JOIN stock   AS s ON s.product_id = p.id
LEFT JOIN reviews AS r ON r.product_id = p.id
WHERE p.is_active
GROUP BY p.id, p.title, c.name, b.name, p.price, s.quantity;
```

Назначение:

Готовая витрина для сайта: активные товары с названием категории и бренда, остатком на складе, средней оценкой и числом отзывов. Таблицы stock и reviews присоединяются через LEFT JOIN, чтобы не потерять товары без отзывов; COALESCE заменяет отсутствующий остаток нулём. Представление содержит 900 строк - по числу активных товаров.

Фрагмент результата:

Команда CREATE VIEW выполнена; обращение к представлению - в запросе 11.

Материализованное представление mv_category_stats

Текст запроса:

```
-- Материализованное представление: сводка по категориям
CREATE MATERIALIZED VIEW mv_category_stats AS
SELECT c.id AS category_id, c.name AS category,
    COUNT(p.id) AS products_count,
    round(AVG(p.price), 2) AS avg_price
FROM categories AS c
LEFT JOIN products AS p ON p.category_id = c.id
GROUP BY c.id, c.name;
```

Назначение:

В отличие от обычного, материализованное представление физически хранит результат запроса и обновляется командой REFRESH MATERIALIZED VIEW mv_category_stats. Чтение из него быстрее, но данные актуальны на момент последнего обновления. Здесь хранится сводка по категориям.

Фрагмент результата:

Команда CREATE MATERIALIZED VIEW выполнена (12 строк по числу категорий); обращение - в запросе 12.

Запрос 10. Обращение к представлению v_brand_revenue

Текст запроса:

```
-- Запрос 10
```

```
SELECT brand_name AS "Бренд", orders_count AS "Заказов", items_sold AS
"Продано, шт.", revenue AS "Выручка"
FROM v_brand_revenue
ORDER BY revenue DESC
LIMIT 5;
```

Назначение:

Пять брендов с наибольшей выручкой. Запрос к представлению не содержит соединений и группировки - они скрыты в его определении.

Фрагмент результата:

	Бренд text	Заказов bigint	Продано, шт. bigint	Выручка numeric
1	Четвероногий др...	212	465	1108642.49
2	Hartz	184	402	1071220.57
3	Pedigree	170	356	1051793.12
4	Trixie	191	400	1030752.43
5	Ferplast	166	372	999059.98

Строк в результате: 5. Совпадает с первыми строками результата запроса 8.

Запрос 11. Обращение к представлению v_product_catalog

Текст запроса:

```
-- Запрос 11
SELECT id, title, category, brand, price, in_stock, avg_score, reviews_count
FROM v_product_catalog
WHERE in_stock > 0 AND reviews_count > 0
ORDER BY avg_score DESC, reviews_count DESC, id
LIMIT 10;
```

Назначение:

Товары в наличии (in_stock > 0), у которых есть отзывы, упорядоченные по средней оценке, затем по числу отзывов. К представлению можно применять обычные WHERE, ORDER BY и LIMIT.

Фрагмент результата:

	id bigint	title text	category text	brand text	price numeric (12,2)	in_stock integer	avg_score numeric	reviews_count bigint
1	9	Лежанка «Active» №9	Лежанки	Pro Plan	1646.51	180	5.00	1
2	12	Мяч «Light» №12	Мячи и фрисби	Hartz	3242.27	140	5.00	1
3	45	Мяч «Premium» №45	Мячи и фрисби	Pro Plan	3019.11	104	5.00	1
4	53	Мяч «Sensitive» №53	Мячи и фрисби	Royal Canin	2879.69	107	5.00	1
5	57	Сухой корм «Light» №57	Сухой корм	Четвероногий др...	3869.34	83	5.00	1
6	67	Влажный корм «Light» №67	Влажный корм	Hartz	1504.78	146	5.00	1
7	72	Сухой корм «Light» №72	Сухой корм	Whiskas	1042.73	54	5.00	1
8	73	Лежанка «Sensitive» №73	Лежанки	Hill's	3963.47	38	5.00	1
9	75	Влажный корм «Premium» №...	Влажный корм	Royal Canin	1701.30	32	5.00	1
10	85	Шампунь «Premium» №85	Шампуни и косметик...	Royal Canin	3202.31	129	5.00	1

Строк в результате: 10. Показаны первые 10 строк.

Запрос 12. Обращение к материализованному представлению

Текст запроса:

```
-- Запрос 12 (материализованное представление)
SELECT * FROM mv_category_stats WHERE products_count > 0 ORDER BY products_count DESC;
```

Назначение:

Чтение сохранённой сводки по категориям: категории без товаров (корневые) отфильтрованы условием products_count > 0. Результат совпадает с запросом 3, но читается из готовой таблицы без повторного вычисления.

Фрагмент результата:

	category_id bigint	category text	products_count bigint	avg_price numeric
1	6	Сухой корм	161	2544.93
2	12	Лежанки	160	2565.43
3	11	Шампуни и косметик...	149	2535.21
4	7	Влажный корм	147	2715.03
5	8	Мячи и фрисби	147	2440.91
6	10	Ошейники и поводки	126	2597.58
7	9	Когтеточки	110	2484.80

Строк в результате: 7. Семь строк.

8. Для каждого запроса приведите его текст, назначение и фрагмент результата.

Заключение

В ходе работы к базе данных «Онлайн-зоомагазин» составлен набор многотабличных и аналитических запросов. Показаны внутренние соединения трёх и четырёх таблиц, самосоединение таблицы категорий и LEFT JOIN с отбором строк без соответствий (товары без отзывов, пользователи без заказов). Выполнены группировка и агрегация, фильтрация групп через HAVING, подзапросы всех видов - скалярный, с IN, с EXISTS и NOT EXISTS, коррелированный и в секции FROM. Сложные запросы разбиты на шаги с помощью CTE, а иерархия категорий обойдена рекурсивным CTE. Наиболее полезные выборки оформлены в виде двух обычных представлений (v_brand_revenue, v_product_catalog) и одного материализованного (mv_category_stats), обращение к которым продемонстрировано. Отдельно рассмотрены различия между COUNT(*) и COUNT(столбец), между WHERE и HAVING, между обычным и материализованным представлением.